Department of Computer Science and Engineering
Premier University

CSE 454: Compiler Construction Laboratory

Experiment 05: A Program to Identify Unique Identifiers from a C Program and Count Their Frequency.

Submitted by:

Name	Shuvra Roy
ID	02222100051011093
Section	C
Session	Fall 2025
Semester	8th Semester
Performance Date	14.02.2026
Submission Date	28.02.2026

Submitted to:
Nazma Akther
Assistant Professor, Department of CSE
Premier University, Chittagong

Remarks

Experiment No: 05

Experiment Name: A Program to Identify Unique Identifiers from a C Program and Count Their Frequency.

Objective: The objective of this experiment is to design and implement a basic lexical analyzer that can detect identifiers from a given C source program and count the frequency of each identifier. The program follows identifier rules such as allowing letters, digits, and underscores, ensuring the first character is a letter or underscore, and excluding C keywords from identifiers. This experiment helps in understanding token recognition and lexical analysis in compiler design.

Algorithm:

Step 1: Start the program by including the necessary header files such as iostream, fstream, map, and cctype. Declare all required variables including file pointers, strings for storing file names and lines, and a map data structure to store identifiers along with their frequency. Also prepare a predefined list of C language keywords so that reserved words are not considered as identifiers during scanning.

Step 2: Initialize an empty map to store identifiers and their frequency count. This data structure allows storing each unique identifier as a key and automatically updating its occurrence count whenever the identifier appears again in the source program.

Step 3: Begin reading the source file line by line using a loop until the end of the file is reached. This ensures that the entire source code is scanned systematically without missing any part of the program.

Step 4: Initialize a boolean flag to detect string literals and a temporary string variable to build words from characters. This flag helps ignore characters that appear inside double quotation marks so that identifiers present inside strings are not incorrectly counted.

Step 5: If the character is a letter, digit, or underscore (_), append it to the temporary word variable. This step helps construct possible identifiers from consecutive valid characters.

Step 6: If a space, punctuation mark, or special symbol is encountered, treat the currently built sequence of characters as a complete word. Then check whether this word satisfies the rules of a valid identifier.

Step 7: Compare the valid word with the list of C language keywords. If the word matches any keyword such as int, while, or return, skip it because keywords cannot be identifiers.

Step 8: If the word is a valid identifier and not a keyword, store it in the map. If the identifier already exists in the map, increase its frequency count by one; otherwise, insert it into the map with an initial frequency of one.

Step 9: Continue this scanning and checking process for every character and every line until the entire source file has been processed completely.

Step 10: After scanning the full source program, write the heading into the output file indicating that the file contains the list of identifiers and their frequencies. Then traverse the map and write each identifier along with its corresponding frequency count in the format:
identifier : frequency.

Step 11: Display the total number of unique identifiers found in the source program on the terminal. Also print a message confirming that the identifier list and frequency information have been successfully saved in the specified output file.

Step 12: Close both the source file and the output file properly to release system resources and avoid file corruption.

Step 13: End the program successfully.

Source Code:

```cpp
#include <iostream>
#include <fstream>
#include <string>
#include <cctype>

using namespace std;
set<string> keywords = {
"auto","break","case","char","const","continue","default","do","double","else",
    "enum","extern","float","for","goto","if","int","long","register","return",
    "short","signed","sizeof","static","struct","switch","typedef","union",
    "unsigned","void","volatile","while"
};
bool isValidIdentifier(const string &word)
{
    if (!(isalpha(word[0]) || word[0] == '_'))
        return false;
    for (char c : word)
        if (!(isalnum(c) || c == '_'))
            return false;
    return true;
}
int main(){
    string inputFile, outputFile, word;
    ifstream source;
    ofstream dest;
    source.open(inputFile);
    dest.open(outputFile);
    if (!source || !dest){
        cout << "Error opening file!" << endl;
        return 1;
    }
    set<string> identifiers;
    dest << "List of Unique Identifiers from file (" << inputFile << "):\n\n";
    while (source >> word){
        word = cleanWord(word);
        if (word.empty())
            continue;
        if (isValidIdentifier(word) && keywords.find(word) == keywords.end()) {
            identifiers.insert(word);
        }
    }
    for (const string &id : identifiers)
        dest << id << endl;
    source.close();
    dest.close();
    cout << "\nTotal Unique Identifiers Found: " << identifiers.size() << endl;
    cout << "Identifier list saved in: " << outputFile << endl;
    return 0;
}
```

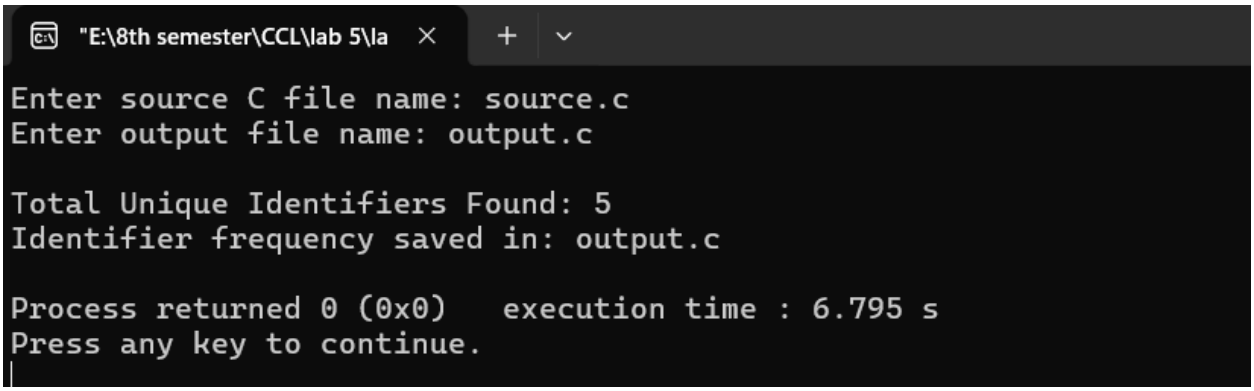
Input: (source.c) file.

```c
#include <stdio.h>

int main()
{
    int number = 10;
    int _value = 20;
    int result = number + _value;

    printf("Result = %d\n", result);
    printf("Name: Shuvra Roy ID: 0222210005101093\n");
    return 0;
}
```

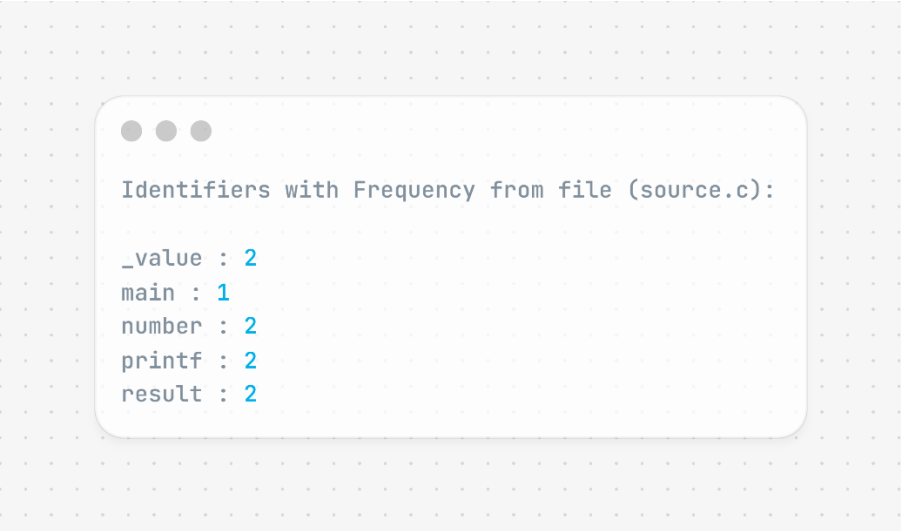
Output:

After execution, the program scans the entire C source file and detects all valid identifiers while excluding keywords and invalid tokens. It also counts how many times each identifier appears in the program and writes the result into the output file.

Sample Terminal Output:



Sample Output File Content: (output.c) file.



Discussion:

This experiment demonstrates the implementation of a basic lexical analyzer to detect identifiers in a C program. The program reads the source file, ignores preprocessor directives and string literals, and identifies valid identifiers according to C language rules. It excludes keywords and counts the frequency of each unique identifier using a map data structure. This experiment provides a clear understanding of lexical analysis, token recognition, and identifier classification, which are essential concepts in compiler design and programming language processing.

Department of Computer Science and Engineering
Premier University

CSE 454: Compiler Construction Laboratory

Experiment 06: Write a program to detect and remove left recursion from a given context-free gram mar using C.

Submitted by:

Name	Shuvra Roy
ID	02222100051011093
Section	C
Session	Fall 2025
Semester	8th Semester
Performance Date	21.03.2026
Submission Date	28.03.2026

Submitted to:
Nazma Akther
Assistant Professor, Department of CSE
Premier University, Chittagong

Remarks

Experiment No: 06

Experiment Name: Write a program to detect and remove left recursion from a given context-free gram mar using C.

Objective: The objective of this experiment is to design and implement a C++ program that detects left recursion in a context-free grammar and removes it by transforming the grammar into an equivalent grammar without left recursion. Eliminating left recursion is important because many top-down parsing techniques used in compiler design cannot handle left-recursive grammars.

Algorithm:

Step 1: Start the program by initializing the necessary variables and including required libraries for input, output, and string processing. Prepare variables to store the grammar production, left-hand side non-terminal, and right-hand side productions.

Step 2: Ask the user to enter a grammar production in the standard format such as S->Sa|b. The program reads this production as a string and stores it for processing.

Step 3: Extract the left-hand side of the production rule. The first character of the production represents the non-terminal symbol. This symbol will be used to check whether left recursion exists.

Step 4: Extract the right-hand side of the production by removing the left-hand side and the arrow symbol (->). The remaining part contains one or more productions separated by the | symbol.

Step 5: Split the right-hand side into individual productions using the | symbol as a separator. Each separated string represents a possible production rule.

Step 6: Analyze each production individually. For every production, check the first character of the string to determine whether it is the same as the non-terminal on the left-hand side.

Step 7: Classify the productions into two groups:

- **Left recursive productions (α)**: Productions that begin with the same non-terminal symbol.
- **Non-left recursive productions (β)**: Productions that begin with a different symbol.

Step 8: Check whether the list of left recursive productions is empty or not.

- If it is empty, then the grammar does not contain left recursion and the program simply prints that no left recursion is found.

Step 9: If left recursion is detected, apply the standard transformation rule used in compiler design to eliminate left recursion. Introduce a new non-terminal symbol (for example A') and transform the grammar according to the following rules:

- **A → βA'**
- **A' → αA' | ε**

where β represents the non-left recursive productions and α represents the left recursive parts.

Step 10: Construct the new grammar using the transformation rule and print the grammar without left recursion. Display the result in a clear format so the user can easily understand the transformation.

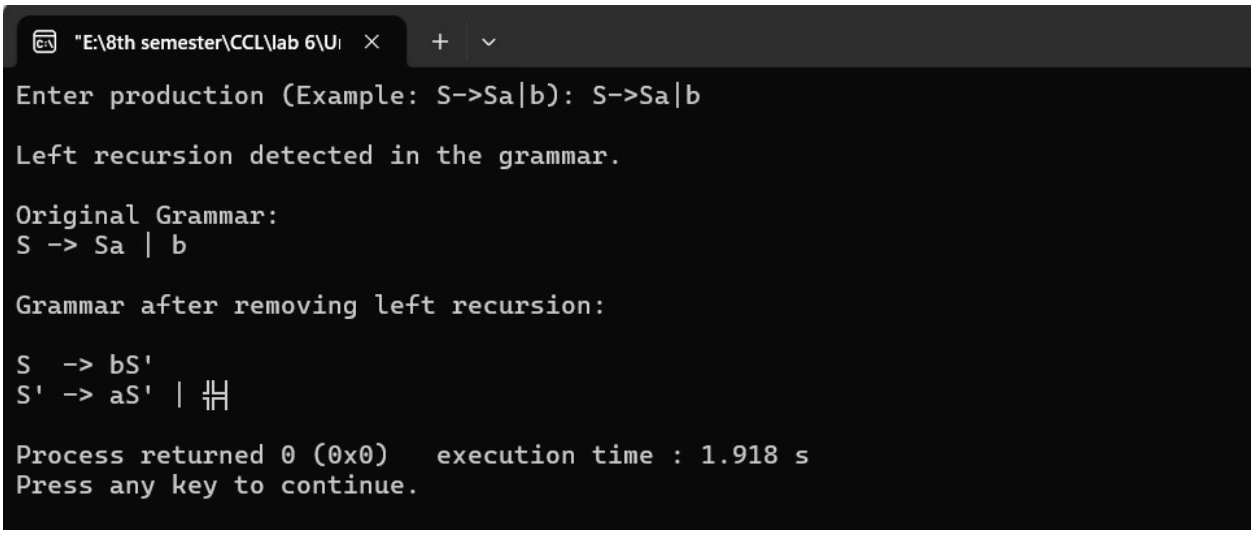
Step 11: Display both the original grammar and the transformed grammar after removing left recursion to show how the grammar has been modified.

Step 12: End the program after printing the final grammar and releasing any used resources.

Source Code:

```cpp
#include <iostream>
#include <string>
#include <vector>
using namespace std;
int main()
{
    string production;
    char nonTerminal;
    vector<string> alpha, beta;
    cout << "Enter production (Example: S->Sa|b): ";
    cin >> production;
    nonTerminal = production[0];
    string right = production.substr(3);
    string temp = "";

    for (char c : right){
        if (c == '|'){
            if (temp[0] == nonTerminal)
                alpha.push_back(temp.substr(1));
            else
                beta.push_back(temp);

            temp = "";
        }
    }
    if (alpha.empty()){
        cout << "No left recursion found in the grammar." << endl;
        return 0;
    }

    for (int i = 0; i < alpha.size(); i++){
        cout << nonTerminal << alpha[i];
        cout << " | ";
    }
    cout << endl;
    cout << "\nGrammar after removing left recursion:\n\n";
    cout << nonTerminal << "  -> ";

    for (int i = 0; i < beta.size(); i++){
        cout << beta[i] << nonTerminal << "'";
        if (i != beta.size() - 1)
            cout << " | ";
    }
    for (int i = 0; i < alpha.size(); i++){
        cout << alpha[i] << nonTerminal << "'";
        if (i != alpha.size() - 1)
            cout << " | ";
    }
    cout << " | ε" << endl;
    return 0;
}
```

Input and output:



Discussion: This experiment demonstrates the concept of detecting and removing left recursion in a context-free grammar. Left recursion occurs when a non-terminal symbol appears at the beginning of its own production rule, which can cause infinite recursion in top-down parsers. The program reads a grammar production, checks whether the right-hand side begins with the same non-terminal, and identifies such productions as left recursive. If left recursion exists, the program applies the standard transformation rule to eliminate it by introducing a new non-terminal symbol and rewriting the grammar rules. This transformation preserves the language generated by the grammar while making it suitable for predictive parsing techniques. Through this experiment, students gain practical knowledge of grammar transformation and the importance of removing left recursion in compiler design.

Department of Computer Science and Engineering
Premier University

CSE 454: Compiler Construction Laboratory

Experiment 07: Write a program to apply Algorithm for Left Factoring a Context-Free Grammar (CFG).

Submitted by:

Name	Shuvra Roy
ID	02222100051011093
Section	C
Session	Fall 2025
Semester	8th Semester
Performance Date	28.03.2026
Submission Date	07.04.2026

Submitted to:
Nazma Akther
Assistant Professor, Department of CSE
Premier University, Chittagong

Remarks

Experiment No: 07

Experiment Name: Write a program to apply Algorithm for Left Factoring a Context-Free Grammar (CFG).

Objective: The objective of this experiment is to implement a program that performs left factoring on a given Context-Free Grammar (CFG). Left factoring is a grammar transformation technique used to eliminate common prefixes from productions. This transformation helps make the grammar suitable for top-down parsing methods such as recursive descent parsing by removing ambiguity in production choices.

Algorithm:

Step 1: Start the program by including the necessary header files and declaring required variables. Prepare variables to store the grammar production, the non-terminal symbol on the left-hand side, and the list of productions on the right-hand side.

Step 2: Ask the user to enter a grammar production in the standard form such as A->abC|abD|aE. Read the input as a string and store it for further processing.

Step 3: Extract the left-hand side of the production rule. The first character of the input string represents the non-terminal symbol (for example A). This symbol will remain the main production symbol in the grammar.

Step 4: Extract the right-hand side of the production by removing the non-terminal and the arrow symbol (->). The remaining part of the string contains multiple productions separated by the | symbol.

Step 5: Split the right-hand side into individual production strings using the delimiter |. Each separated string represents one possible production rule of the grammar.

Step 6: Store all extracted productions into a vector or list data structure. This allows the program to easily process and compare each production.

Step 7: Compare the productions to determine whether a common prefix exists among them. The program examines characters from the beginning of each production and finds the longest sequence of characters that appear at the beginning of multiple productions.

Step 8: If a common prefix is found, store this prefix as α. The remaining part of each production after removing the prefix is considered the suffix and is represented as β.

Step 9: Apply the left factoring rule used in compiler design. Introduce a new non-terminal symbol (for example A') and rewrite the grammar as follows:

- $A \rightarrow \alpha A'$
- $A' \rightarrow \beta 1 \mid \beta 2 \mid \ldots \mid \beta n$

where α represents the common prefix and β represents the remaining parts of the productions.

Step 10: Construct the new grammar rules using the prefix and suffix values. Ensure that empty suffixes are replaced with the epsilon symbol (ε) to represent an empty string.

Step 11: Display the transformed grammar clearly so that the user can understand how the original grammar has been modified after applying left factoring.

Step 12: If no common prefix is found among the productions, display a message stating that left factoring is not required for the given grammar.

Step 13: End the program successfully after printing the final grammar rules.

Source Code:

```cpp
#include <iostream>
#include <vector>
using namespace std;

string prefix(string a,string b){
    string p="";
    for(int i=0;i<min(a.size(),b.size()) && a[i]==b[i];i++)
        p+=a[i];
    return p;
}

int main(){
    string g,t="";
    cout<<"Enter production (like: A->abC|abD|aE): ";
    cin>>g;

    char A=g[0];
    vector<string> r;

    for(int i=3;i<g.size();i++){
        if(g[i]=='|'){ r.push_back(t); t=""; }
        else t+=g[i];
    }
    r.push_back(t);

    string p=r[0];
    for(int i=1;i<r.size();i++)
        p=prefix(p,r[i]);

    if(p==""){ cout<<"No Left Factoring needed\n"; return 0; }

    cout<<"\nAfter Left Factoring:\n\n";
    cout<<A<<" -> "<<p<<A<<"'";

    for(string s:r)
        if(s.substr(0,p.size())!=p)
            cout<<" | "<<s;

    cout<<"\n"<<A<<"' -> ";

    for(int i=0;i<r.size();i++){
        string suf=r[i].substr(p.size());
        cout<<(suf==""?"ε":suf);
        if(i!=r.size()-1) cout<<" | ";
    }
}
```

Input:

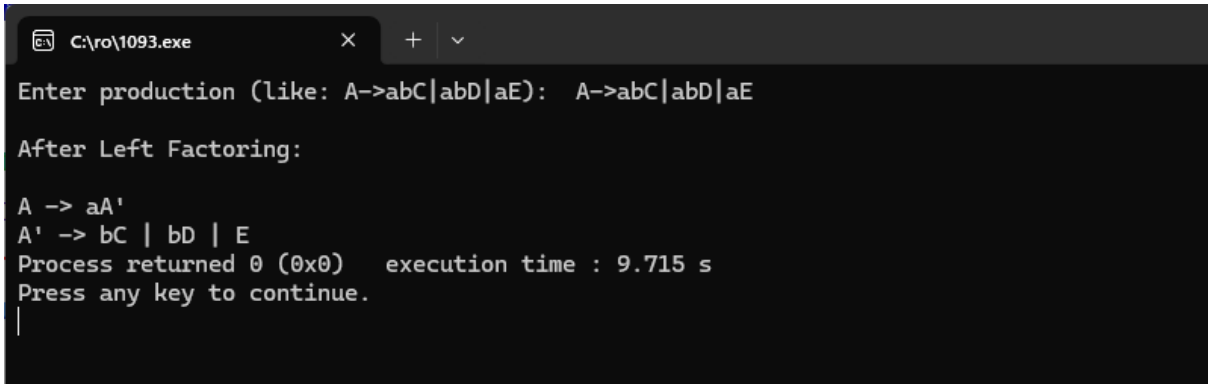
The program takes a Context-Free Grammar (CFG) production from the user. The grammar is entered in the format:

A->abC|abD|aE

Here, A is the non-terminal and the productions are separated by the symbol |.

Output:

After running the program, it applies the left factoring algorithm to the given grammar and displays the transformed grammar.



This shows the grammar after removing the common prefix.

Discussion:

This experiment demonstrates the implementation of the left factoring technique used in compiler design. Left factoring is important because it removes ambiguity in grammar productions that share a common prefix. Such ambiguities make it difficult for predictive parsers to determine which production rule to use during parsing. By factoring out the common prefix and introducing a new non-terminal symbol, the grammar becomes more suitable for top-down parsing techniques such as recursive descent parsing. Through this experiment, students gain a practical understanding of grammar transformation and the role of left factoring in syntax analysis.